LAYOUT

Layout'un Faydaları

Layout Oluşturma

Layout Kullanımı

Layout Çalışma Mantığı

RenderSectİon Fonksİyonu

RenderSection Kullanımı

RenderSection Kullanım Alanları

_VİEWSTART DOSYASI

_ViewStart Dosyasi Oluşturulmasi

_ViewStart Dosyasi Kullanimi

_VİEWIMPORTS DOSYASI

_ViewImport Dosyasi Oluşturulmasi

_ViewImport Dosyasi Kullanimi

Layout

Layout’un Faydaları

Uygulamada sabit kalan, her sayfada kullanılan nav bar gibi yapıların tekrar tekrar yazılmasını önler. Yani kod tekrarının önüne geçer.

Sadece Nav bar değil her sayfada tekrar edecek tüm yapıları tek yerden yönetilebilen bir forma sokar.

Layout sadece Asp.Net Core veya Backend’e özgü değildir, Frontend mimarilerinde de Layout yapısı vardır.

Layout Oluşturma

Layout bir “.cshtml” dosyasıdır.
Layout herhangi bir dosyanın içerisinde olabilir.

Ancak genellikle Views klasörü altındaki “Shared” klasörü altında oluşturulur. Raconsaldır.

Shared klasörü altında “_” ile başlayan bir .cshtml dosyası oluşturulur.
Layout aslında bir View dosyasıdır.

Layout dosyaları “_” ile başlar. Raconsaldır.

Layout Kullanımı

Layout sayfasında, tekrar etmeyen / yani asıl içeriğin kendisi olan / sayfadan sayfaya değişecek olan içeriğin geleceği yere RenderBody() fonksiyonu yazılır. Bu sayfaya göre değişecek olan içeriğin geleceği yeri belirtir.

Layout’u kullanacak olan View’lere ilgili Layout’u belirtmek gerekir.

Bunun için View dosyalarının içerisine Razor sytanx’ı ile Layout = “ilgili Layout Path’i” yazılır.

```
@{
    Layout = "~/Views/Shared/_Layout.cshtml";
}

Burası Sayfa 1
```

Layout Çalışma Mantığı

Önce ilgili **View (.cshtml)** çalıştırılır ve ürettiği içerik doğrudan ekrana basılmaz, geçici olarak tutulur. Ardından View içinde belirtilmiş olan **Layout** dosyası devreye girer ve render edilmeye başlanır.

Layout içerisindeki @RenderBody() çağrıldığı noktada, daha önce View tarafından üretilmiş olan içerik bu alana yerleştirilir.

Eğer Layout kullanılıyorsa kesin olarak RenderBody kullanılmak zorundadır.

RenderSection Fonksiyonu

RenderBody'nin özelleşmiş hali gibi düşünülebilir. Layout'ta sadece belirli sayfalarda belirli veriler göstermek isteniyorsa kullanılabilir. Örneğin ürünler sayfasına girince Nav bar'da popüler ürünler seçeneği çıkması gibi.

RenderSection Kullanımı

RenderBody gibi Layout içerisinde tanımlanır fakat RenderBody'nin aksine bir den fazla tanımlanabilir.

Layout'da kullanılmak istenen yere RenderSection yazılır. İki parametre alabilir. İlk parametre kullanılacak isim, ikinci parametre ise default olarak True gelen Required durumu. Eğer bu Required false yapılmazsa her View

içerisinde Bir RenderSection tanımlaması yapılmasını bekler eğer her View'de bu tanımlama yapılmazsa uygulama patlar.

```
@RenderSection("kampanya", required: false)
```

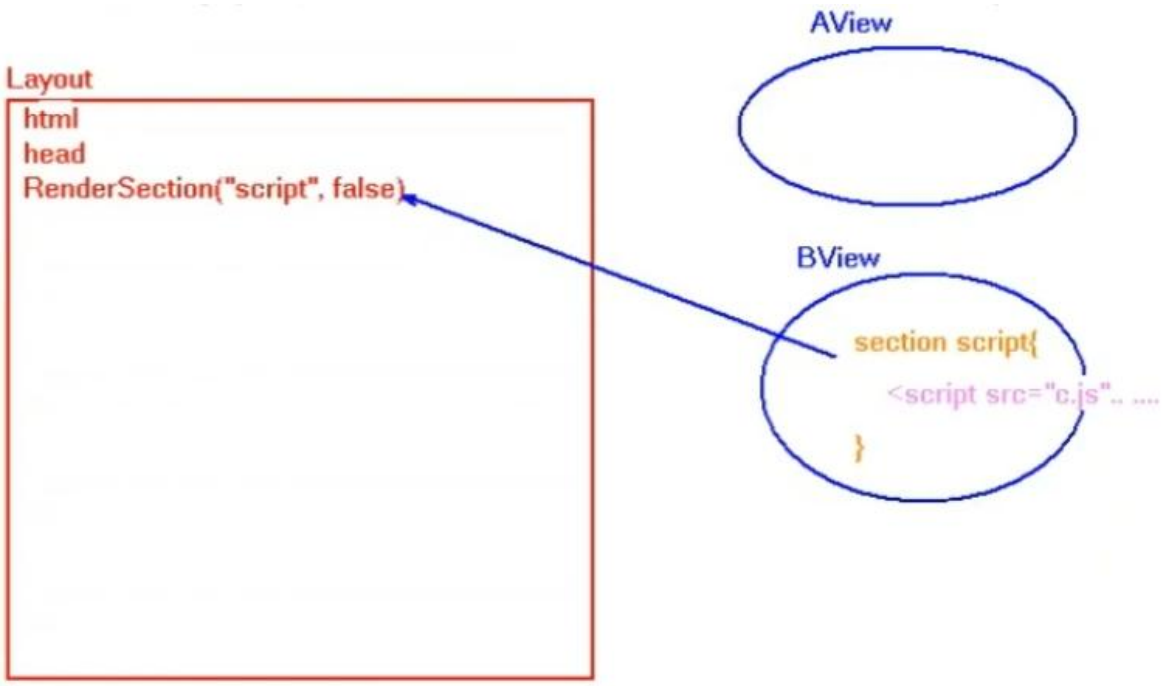
View içerisinde ise yine Razor syntax ile ilgili RenderSection ismi belirtilir ve Scope içerisine gerekli veriler yazılır.

```
@section kampanya {
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="" asp-controller="Layout" asp-action="Kampanya">Kampanyalar</a>
    </li>
}
```

View içerisinde Section'a herhangi bir noktadan veri gönderilebilir. İlla sayfanın üstünden tanımlanacak diye bir şey yok.

RenderSection Kullanım Alanları

Genellikle Javascript kütüphanelerini sayfalara verirken kullanılıyor. Her sayfa her javascript kütüphanesine ihtiyaç duymuyor. Bu sebeple sadece gerekli Javascript kütüphanelerini vermek için kullanılıyor.



MVC için hala kullanışlı olsa da Api tarafında bir karşılığı yoktur.

_ViewStart Dosyası

Tüm View'lerin atası denilebilir.

Bir uygulamada önce ViewStart render edilir ardından ilgili View'lar render edilir.

Tüm View'lerin ortak kullanacağı Layout tanımlaması bu dosya içerisinde yapılır.

Buradaki amaç, normalde Layout kullanıldığında her View'e ilgili Layout'u tanımlamak gerekir. Bu tekrara düşmeye neden olur. Bunun yerine bu Layout'ları tek bir yerden tanımlamak için ViewStart dosyası kullanılır.

_ViewStart Dosyası Oluşturulması

View klasörü altında _ViewStart.cshtml olarak oluşturulması gerekir. Aksi halde mimari tanıyamaz. Yani raconsal değil zorunludur.

_ViewStart Dosyası Kullanımı

ViewStart dosyası içerisine yerleştirilen Layout Path'i tüm View'ler için geçerli olur. Bu aşamadan sonra View'lerde Layout tanımalaması yapamaya gerek yoktur.

Eğer bir View'de, ViewStart'da tanımlanan Layout kullanılmak istenmiyor olabilir veya başka bir Layout kullanmak isteniyor olabilir.

Layout kullanılmak istenmiyorsa ilgili View içerisinde Layout'a Null verilir.

Eğer başka bir Layout kullanılmak isteniyorsa ilgili View içerisine istenen Layout verilerek ViewStart içerisindeki Layout ezilir.

_ViewImports Dosyası

Razor sayfaları için kütüphane ve namespace tanımlamalarını tek tek her sayfaya farklı tanımlamaktansa ortak olarak tanımlanmasını sağlayan bir dosya.

_ViewStart dosyasından farkı, ViewStart daha çok Html, Css ve JavaScript dosyalarını ortak tanımlarken _ViewImport dosyası kütüphane, namespace gibi programatik yapıları tanımlamak için kullanılır.

Örneğin bir View model olarak Product alacak fakat model olarak alması için tüm namespace'i vermek gerekiyor. Örn Asp.Net-Core-Temel.Entiteies.Product gibi bunun yerine eğer bu namespace _ViewImport içerisinde tanımlanırsa View içerisinde sadece Product yazmak yeterli olacaktır.

_ViewImport Dosyası Oluşturulması

View klasörü altında _ViewImports.cshtml adı ile oluşturulur.

_ViewImport Dosyası Kullanımı

ViewImport dosyasının içerisine ilgili namespace using ile verilir.

Örneğin Layout_Yapilari.Entities

Bu aşamadan sonra View'ler içesisinde Entities içerisindeki tüm Entity'ler direkt olarak erişilebilir olur. View içerisinde model Product vermek yereli olur.

Ayrıca MVC projesi oluşturulunca otomatik olarak bu dosyada _Layout ve TagHelper için gerekli kütüphaneler de eklenir.